

The $\Omega = 1$ rule under Adam, beside its prior art, and against a well-tuned constant

Status. This is a note, not evidence (R8). It was written on 2026-09-23 at the owner's request (prompt-log entry 104).

Sources. Every number is printed by a committed script and pinned beside it:

- Continuous_Learning/checks/adam_checks.txt (A1-A8);
- adam_checks_2.txt (B1-B3);
- adam_checks_3.txt (B4-B5);
- runs/eq4/score.txt (ledger rows EQ4-*).

Declarations. Each battery was declared before it ran: DECLARATION_ADAM.md (pushed 2026-09-23T04:54:53Z), DECLARATION_ADAM_2.md (05:01:47Z) and DECLARATION_ADAM_3.md (05:03:32Z).

Evidence level. The synthetic checks sit at rung R4 (declared on a synthetic world). The EQ4 rows are ledger rows, weakly anchored. The literature is recorded in docs/citations/continual_learning_2026-09-23.md.

In plain words. In plain words. We asked four questions about the balancing rule. What happens with the optimiser most big models use? Is the rule really new? Where did its good results come from? And does it beat simply picking the best fixed setting by careful search? The short answers are these. With that optimiser (Adam) the rule has nothing to fix. The rule is almost exactly a trick published in 2020. Its good results came mostly from comparing it with a setting that was searched too roughly. And a carefully searched fixed setting does as well or better everywhere we looked. What is left is real but small: the rule finds a safe setting without any search.

1. The answer first

1. Under Adam (the past term inside the optimiser) a fixed weight is already scale-robust, so the rule has nothing to do.

- A1: Adam's trajectory is unchanged when the whole gradient is rescaled. The largest deviation over scales 1/16 to 256 is 8.667e-09. - A2: no fixed weight on the grid diverges at any scale. - A3: the fixed weight tuned at $c = 1$ stays within 10 % of the Adam-tuned weight at every c . Its totals are 1.9094, 3.2859, 3.4403 and 3.4486 against the tuned 1.9094, 3.2859, 3.4399 and 3.4470.

1. Under plain SGD the rule's step bound keeps it finite at every scale, but a well-tuned constant beats it where the scales differ.

- B5, with a fine grid of 60 weights: at $c = 16$ the rule's total is 1.186 of the finely tuned constant's without smoothing and 1.490 with the registered smoothing. At $c = 256$ it is 1.423 and 2.073. - Where the scales match, the rule ties (0.997 to 1.050). - The advantage A4 recorded at $c = 16$ (0.660) was measured against a 10-point grid. That grid has no point between 0.03 and 0.1, while the best constant is 0.064222.

1. Without its smoothing the rule is the VQGAN adaptive weight (Esser, Rombach and Ommer, 2020).

- B1: at $c = 16$ the unsmoothed rule totals 7.7432 and the VQGAN-style ratio 7.7434. At $c = 256$ they total 78.3521 and 78.3522. - The registered smoothing (EMA 0.9) costs at every scale (B1) and at every noise level tried (B2: 73.5962 against 7.1299 at noise 0.1, 9.7260 against 7.7432 at 0.5, 12.5881 against 9.5821 at 1).

1. Under heavy-ball momentum (the usual optimiser for vision) the rule is finite but behind the tuned constant at every scale (A8: 1.015, 1.077, 1.796, 4.788).

2. On real data, under plain SGD, the bounded rule EQ-B fails EQ4-1 on 2 of 6 unseen carriers.

- Under poison, a fixed weight with the same clip is enough (EQ4-3 FAIL, EQ4-4 INERT). - The ER-sum control is violated (EQ4-5). - Details are in reports/eq4.md.

In plain words. In plain words. Imagine two people pulling a cart in different directions, and you set how hard the second one pulls. The rule says "pull exactly as hard as the first person". That never tips the cart over, which is its one virtue. But if you take the time to find the best pull by trying many settings, you find a better one. With the optimiser most language models use, the cart cannot tip over anyway.

2. Why the rule works where it works

The step bound. The rule sets $w = \Omega \|\hat{g}_p\| / \|\hat{g}_q\|$ (EMAs of the present and past gradients). The past pull in the smoothed gradients is then exactly Ω times the present pull, whatever units the past term carries. This is the only property that survives every test here:

- under SGD the rule is finite at every scale (A7);
- the unsmoothed rule and the VQGAN-style ratio share it (A7, B1).

Why a fixed weight needs it under SGD. A fixed weight λ has a stability edge, $\lambda^* = (2/\eta - h_{\max})/f_{\max}$. It moves as $1/c$ when the past term's units change: A2 gives 20.1198 at $c = 1/16$, 1.2575 at 1, 0.0786 at 16 and 0.0049 at 256. A weight tuned in one set of units can therefore diverge in another. At $c = 16$ the weight 1 diverges, and so does the $c = 1$ tuned weight. Heavy-ball momentum moves the edge but keeps it (A8: 398.5476, 24.9092, 1.5568, 0.0973). The edge predicts exactly which grid points diverge at every scale.

Why Adam does not need it. Adam divides each coordinate of its update by the root of a running mean of squared gradients. A rescaled gradient therefore gives the same step (A1), and a large λ changes the direction of the step but not its length. There is no edge to cross (A2), and the fixed weight transfers across scales (A3). The same reason explains the poison result: under Adam a batch 1000 times too large is absorbed by the second moment. The fixed weight's poisoned total is 3.2774 against 3.2859 clean (A6), with no clip needed.

Where Adam leaves room for the rule. The rule has room only when the past pull is applied outside Adam's preconditioner, as AdamW applies weight decay (Loshchilov and Hutter). In that decoupled position A5 finds the following:

- the fixed weight's edge returns (at $c = 16$ every weight from 0.1 up diverges);
- the decoupled rule is finite at every scale;
- its totals are 2.0169, 3.3225, 3.4622 and 7.3244 against the decoupled tuned weight's 2.2137, 3.3356, 3.5385 and 4.4454.

So the decoupled rule is lower at three scales and higher at $c = 256$. That tuned weight comes from the coarse grid, so the B5 caution applies to it too.

3. Why it does not beat a constant

On a stationary problem the rule's weight settles, and a settled weight is a constant.

- **The settled weight (B3).** The median derived w over the last 1000 steps is 0.0480 (no smoothing) and 0.0446 (EMA 0.9) at $c = 16$, and 0.0030 and 0.0027 at $c = 256$. For the smoothed rule a fixed weight at that value gives nearly the same total (B4 REDUCES at every scale).
- **Where the best constant sits.** It lies just under the edge (0.064222 against 0.0786 at $c = 16$; 0.0046713 against 0.0049 at $c = 256$).
- **Why the rule settles lower.** Equal pull is a balance point, not the largest stable pull. The total loss asks for more weight on the past term than equal pull gives when the past term's units are large.
- **Where the unsmoothed rule stands.** It does not reduce at $c = 16$ and $c = 256$: its fixed-weight equivalents total 8.5339 and 92.0368 against the rule's 7.7432 and 78.3521. So its step-to-step variation does some work there, but not enough to reach the finely tuned constant.

What the rule's value is, stated at its full size. Under SGD it finds a stable weight without a search, in any units of the past term. It never beats a searched constant here, and under Adam the search is unnecessary anyway. Any technical effect has to rest on a reduced tuning cost, not on higher accuracy. A tuning-cost claim needs its own test (compute to reach within a stated margin of the best), which has not been run.

In plain words. In plain words. The rule is like a thermostat that is never dangerous but always a little too cautious. A person who tests many settings finds a warmer, better one. The thermostat's value is that nobody has to do the testing.

4. How the rule differs from existing approaches

approach	what sets the weight	same as the rule?	what differs
VQGAN adaptive weight (Esser, Rombach, Ommer 2020; <code>`taming/modules/losses/vqperceptual.py`</code>)	$\ \nabla_{\text{rec}}\ / (\ \nabla_{\text{GAN}}\ + 1e-4)$, clamped to $[0, 1e4]$, at the decoder's last layer, every step	yes, without smoothing (B1: 7.7432 against 7.7434 at $c = 16$)	the rule smooths with an EMA (which costs here), takes the whole parameter vector, and is used for a past term in continual learning
PINN learning-rate annealing (Wang, Teng, Perdikaris 2021)	"gradient statistics" balancing the terms of a composite loss	the same family (formula not read, so not compared)	physics-informed networks, not continual learning
GradNorm (Chen et al. 2018)	learned task weights that equalise gradient norms	the rule is GradNorm at $\alpha = 0$ without the learning (<code>`CROSS_VERIFICATION.md`</code> §4)	GradNorm learns the weights with an extra loss
FedKACE gradient-balanced replay (arXiv:2601.19788, 2026)	a ratio of squared gradient norms, replay against new data, per epoch	the same idea in continual learning [search rendering, direction unverified]	squared norms, per epoch, federated; published January 2026
MEGA-I (Guo et al. 2020)	a ratio of losses	no	losses, not gradients; in EQ4 it is ahead of EQ-B by a step on three carriers
AutoClip (Seetharaman et al. 2020)	clip at a percentile of the gradient-norm history	the prior art for EQ-B's clip	EQ-B clips at $\kappa \times$ the window maximum; EQ4-3 shows the clip carries the poison result with a fixed weight
AdamW (Loshchilov, Hutter 2019)	decay applied outside Adam	the placement A5 uses	AdamW decouples a fixed decay; the rule would decouple a ratio-set pull
Adam itself (Kingma, Ba 2015)	per-coordinate normalisation	makes the rule unnecessary (A1–A3)	—

What is distinct to the rule. Three things:

1. The EMA of gradient vectors. It costs on this model: the best smoothing is 0 at every scale (B1).
2. Its placement as the weight of a past term in continual learning. FedKACE is a close published relative, and the gradient-ratio form is VQGAN's.
3. Its derivation within CRR, as equanimity at $\Omega = 1$.

The step bound is a property of the family, not of the rule.

In plain words. In plain words. Other people had already invented "make the two pulls equal". A well-known image generator does exactly this, and a 2026 paper does it for continual learning. The rule adds a smoothing step, and on our tests the smoothing makes it worse, not better.

5. What this changes

- **Continual learning.** The Ω rule is not an accuracy method on any evidence here. Its standing claim is tuning-free stability under SGD-family optimisers, and that claim is shared with the gradient-ratio family.

- A further real-data study of the rule's accuracy (an "EQ5") is not proposed. The synthetic mechanism predicts that it loses to a finely tuned constant. The real-data record (EQ3, EQ4) is mixed and fragile, with both controls violated, and shows no margin that survives them. - A study that would test what is left is a tuning-cost study: compute to reach within a stated margin of the best constant, against a grid search, under SGD, with the VQGAN ratio as a baseline. It has not been designed and is listed here as the one open item.

- **The patent (information, not legal advice).**

- The weight's form was published in 2020 (VQGAN), before the owner's August 2025 filing. A claim to the ratio as such would meet that prior art. - A technical effect would have to be a reduced tuning cost on a specific technical system. It could not be accuracy. - The owner's application is described as "CRR as a physical systems processor". Whether it claims the Ω rule at all is not known here.

- **The money** (Alexander Plan/ALEXANDER_PLAN.md §11). That estimate priced adoption of the rule if its SGD advantage held at scale. The checks above say its advantage over a well-tuned constant is absent even on the synthetic model, and its form is prior art. The model's inputs are not changed by this note; that is the owner's decision. The dossier carries a dated addendum pointing here.
- **Earlier ratios.** The "ratio to the tuned weight" figures in theory/checks/omega_vs_methods.txt, omega_reprocessed.txt and CROSS_VERIFICATION.md use the same 10-point grid. Where the best constant lies between grid points, they overstate the rule. Those pinned outputs are not edited (R8); this section is the correction.

6. Reproduce

```
uv run python Continuous_Learning/checks/adam_checks.py > Continuous_Learning/checks/adam_checks.txt
uv run python Continuous_Learning/checks/adam_checks_2.py > Continuous_Learning/checks/adam_checks_2.txt
uv run python Continuous_Learning/checks/adam_checks_3.py > Continuous_Learning/checks/adam_checks_3.txt
```

All three are deterministic and CI-checked (scripts/check_all.sh).